What is UML and how can it be used in connection with object-oriented programming

Week 36

Katerina Mangaroska

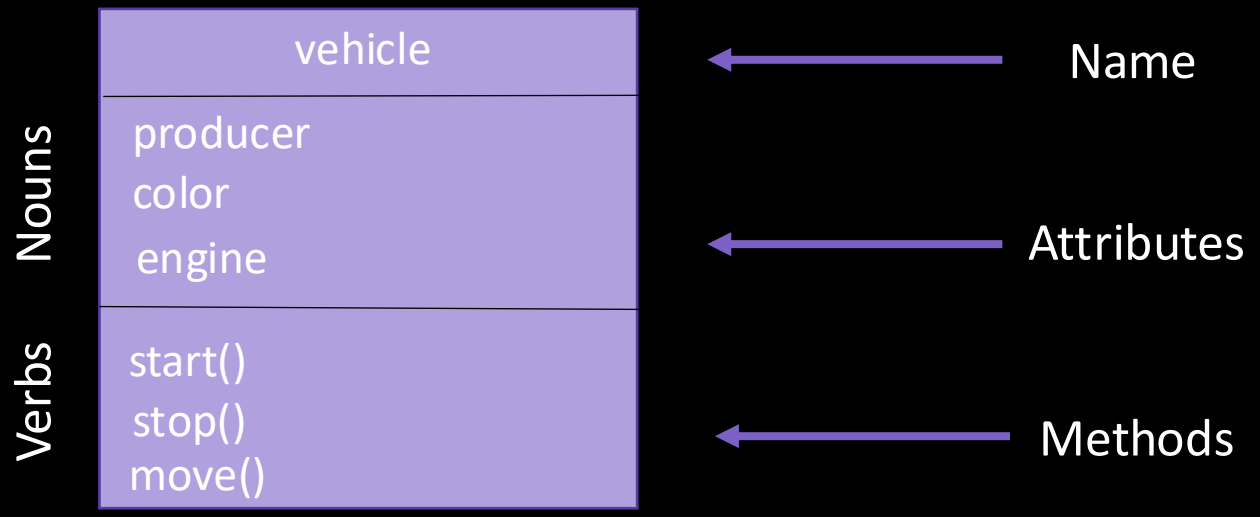
Objectives

- To understand the main concepts behind UML and OOP
- To understand why & how UML can be used with OOP

- Structure of the lecture:
  - What is UML?
  - What is OOP?
  - The object-oriented fundamental:
    - Classes and objects
    - A PIE -- the four pillars of OOP: Abstraction, Polymorphism, Inheritance, and Encapsulation.

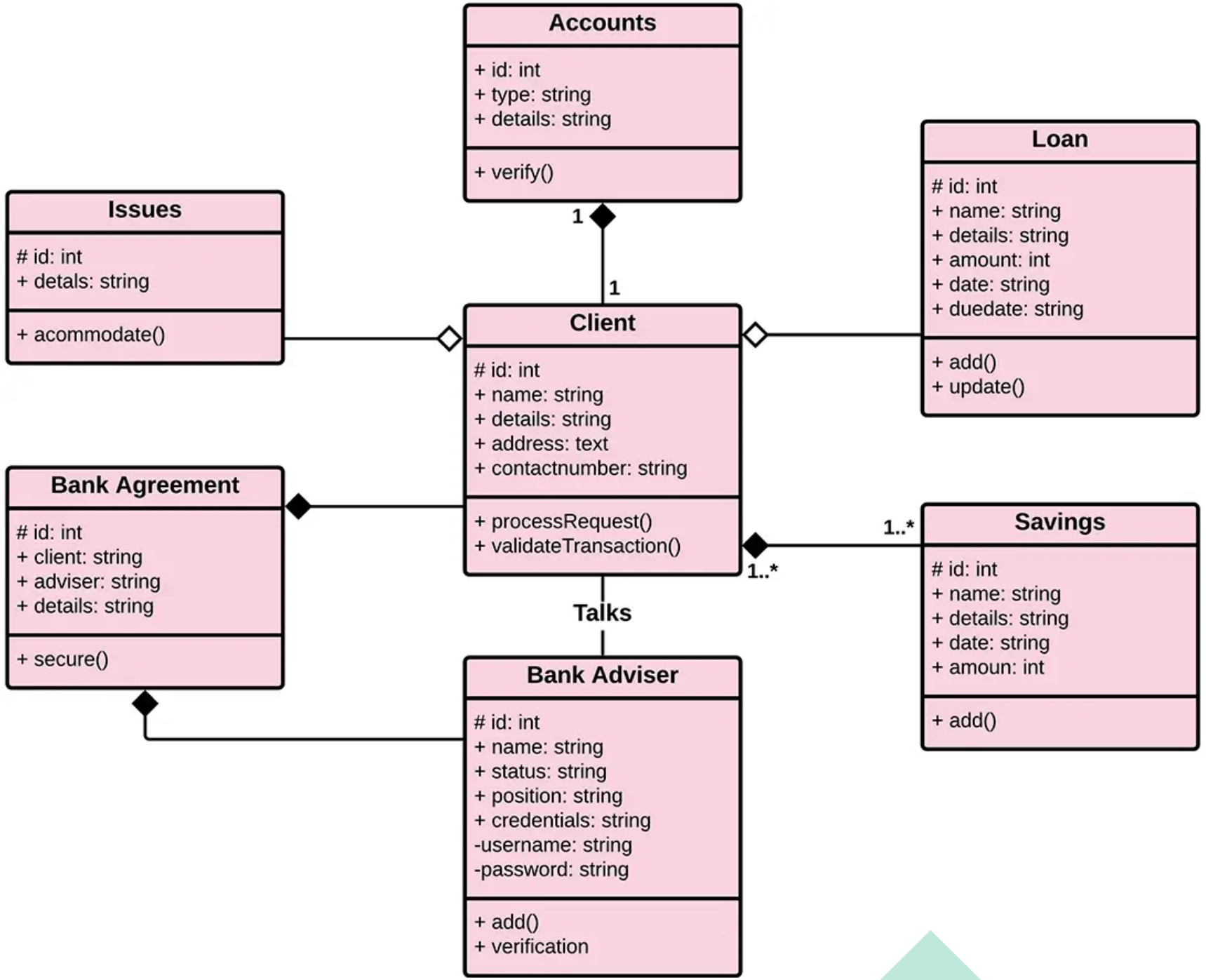
What is UML?

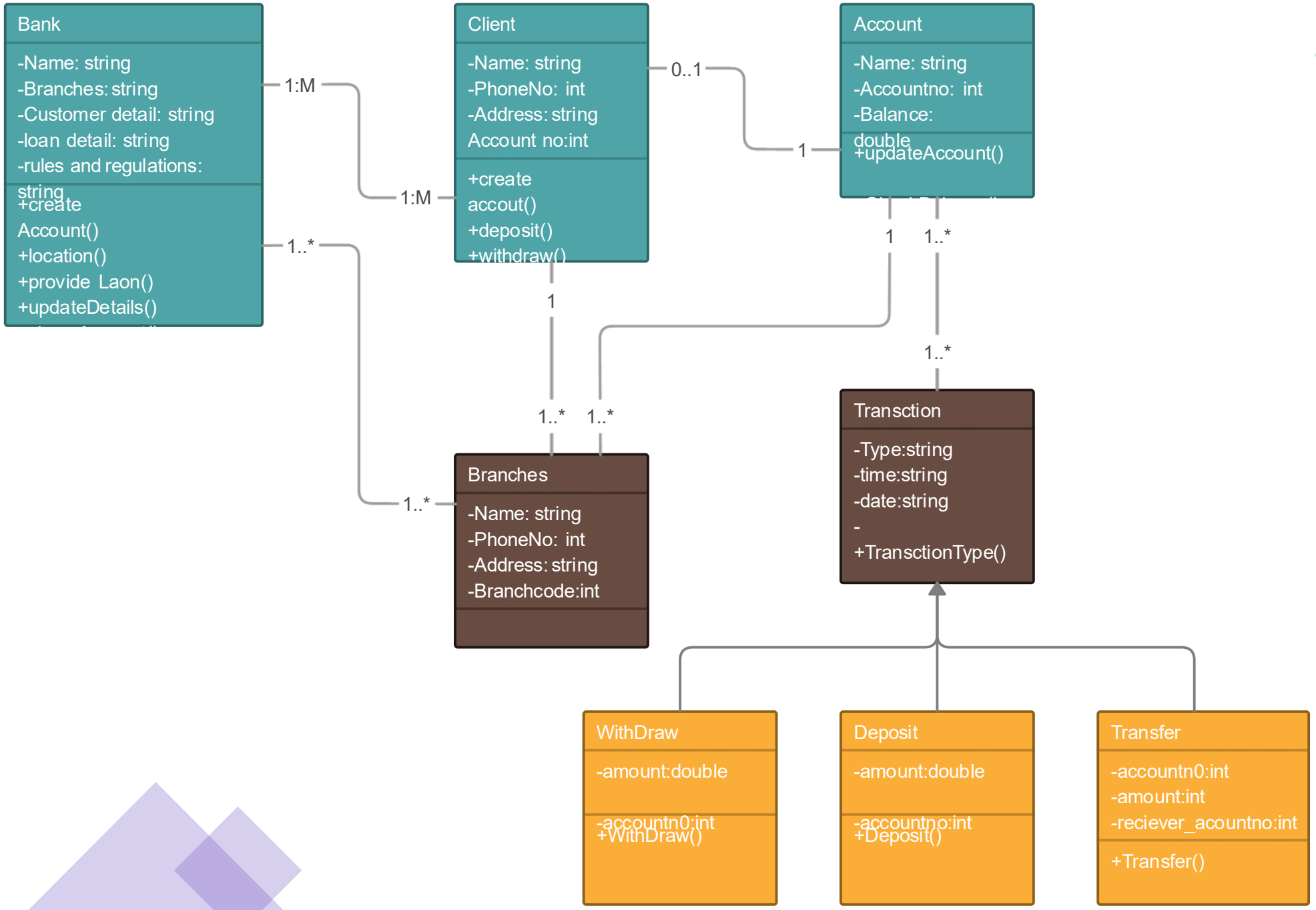
- Unified Modeling Language
- not a programming language. It's a standardized graphical notation for drawing diagrams to visualize object-oriented systems.



Class diagram

- models, i.e., abstract representations of a real-world objects


Accounts
+ id: int
+ type: string
+ details: string
+ verify()
Issues
id: int
+ detals: string
+ acommodate()
Loan
id: int
+ name: string
+ details: string
+ amount: int
+ date: string
+ duedate: string
+ add()
+ update()
Client
id: int
+ name: string
+ details: string
+ address: text
+ contactnumber: string
+ processRequest()
+ validateTransaction()
Bank Agreement
id: int
+ client: string
+ adviser: string
+ details: string
+ secure()
Savings
id: int
+ name: string
+ details: string
+ date: string
+ amoun: int
+ add()
Talks
Bank Adviser
id: int
+ name: string
+ status: string
+ position: string
+ credentials: string
-username: string
-password: string
+ add()
+ verification
1
1
1..*
1..*

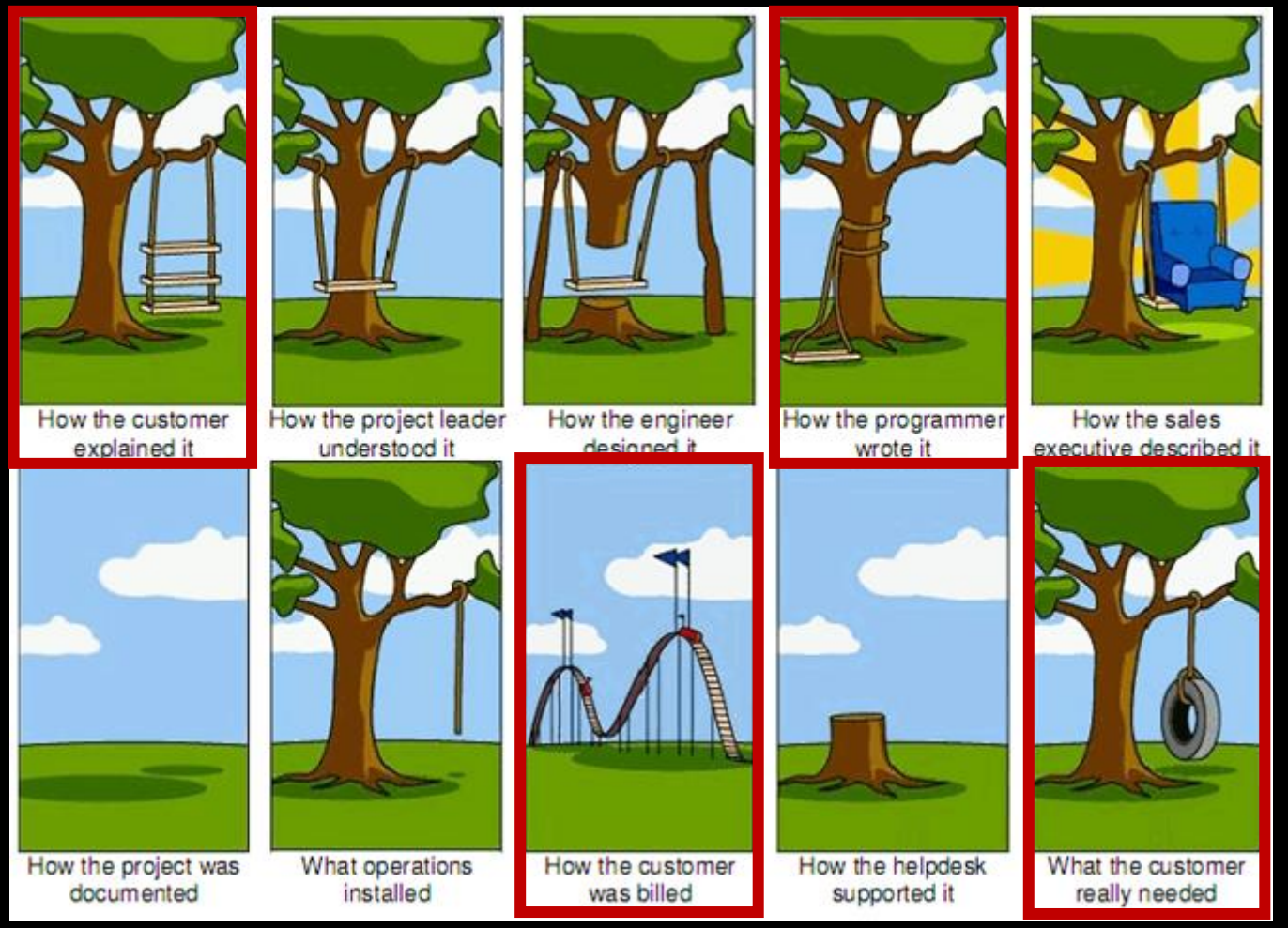

Bank
-Name: string
-Branches: string
-Customer detail: string
-loan detail: string
-rules and regulations: string
+create Account()
+location()
+provide Laon()
+updateDetails()
Client
-Name: string
-PhoneNo: int
-Address: string
Account no:int
+create accout()
+deposit()
+withdraw()
Account
-Name: string
-Accountno: int
-Balance: double
+updateAccount()
Branches
-Name: string
-PhoneNo: int
-Address: string
-Branchcode:int
Transction
-Type:string
-time:string
-date:string
-
+TransctionType()
WithDraw
-amount:double
-accountn0:int
+WithDraw()
Deposit
-amount:double
-accountno:int
+Deposit()
Transfer
-accountn0:int
-amount:int
-reciever_acountno:int
+Transfer()
1:M
1:M
0..1
1
1..*
1
1..*
1..*
1..*
1..*
1..*

Software development projects

Why modeling is a critical step?

index.js ● {...} User Settings

```
1
2  let baseSalary = 30_000;
3  let overtime = 10;
4  let rate = 20;
5
6  function getWage(baseSalary, overtime, rate) {
7    return baseSalary + (overTime * rate);
8  }
9
10 let employee = {
11   baseSalary: 30_000,
12   overtime: 10,
13   rate: 20,
14   getWage: function() {
15     return this.baseSalary + (this.overtime * this.rate);
16   }
17 };
18 employee.getWage();
```








*Please be careful when you put revision clouds on your drawings,
some of the contractors do not understand. *

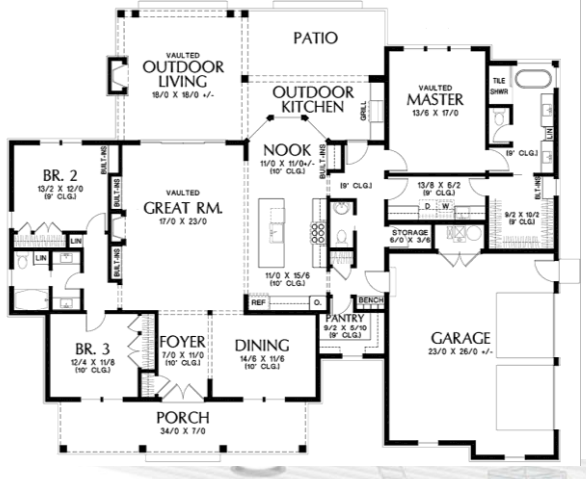




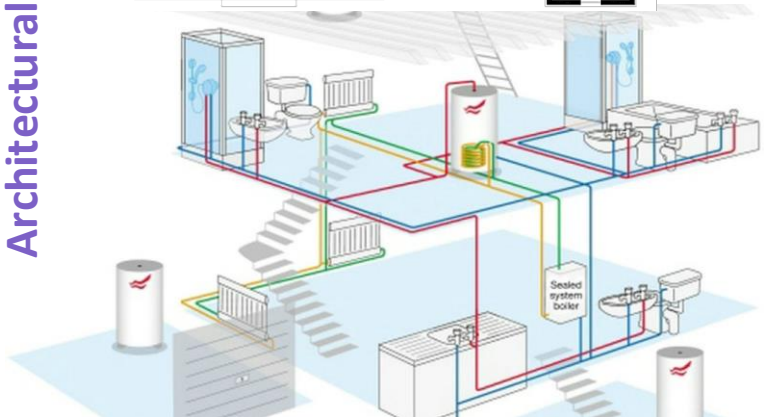


MODEL THE LOGIC AND THE STRUCTURE OF A SYSTEM





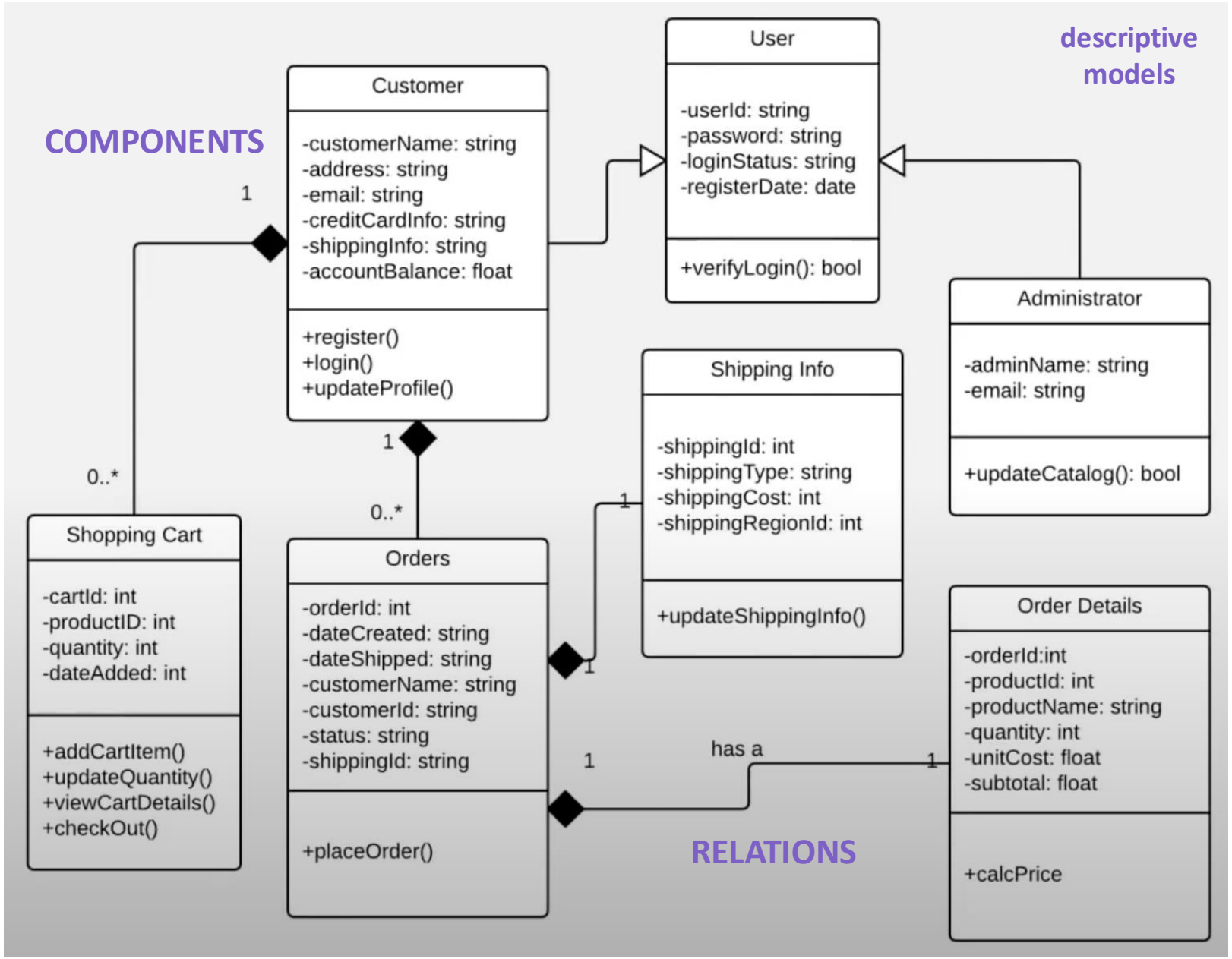
Blueprints



Architectural layouts

descriptive models

COMPONENTS



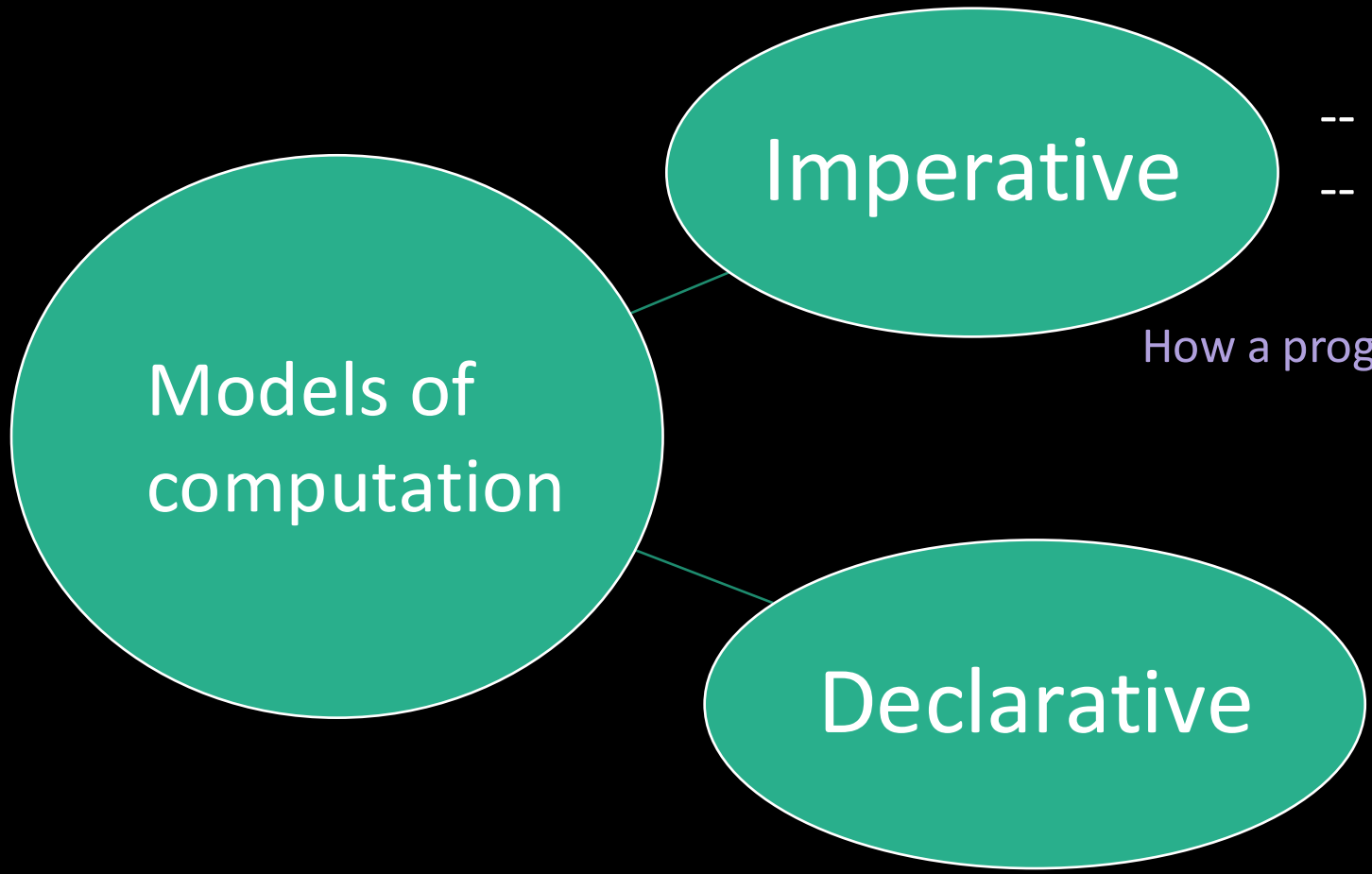
RELATIONS

All good projects (software) start with a great design!

What is a programming paradigm?

- different ways or styles in which a given program or programming language can be organized
- programming paradigms are not languages or tools. You can't "build" anything with a paradigm.
- "multi-paradigm" languages (e.g., Python)
- programming paradigms aren't mutually exclusive (you could use practices from different paradigms at the same time)
- Why? - - certain paradigms are better suited for certain types of problems

Popular Programming Paradigms



Imperative PP

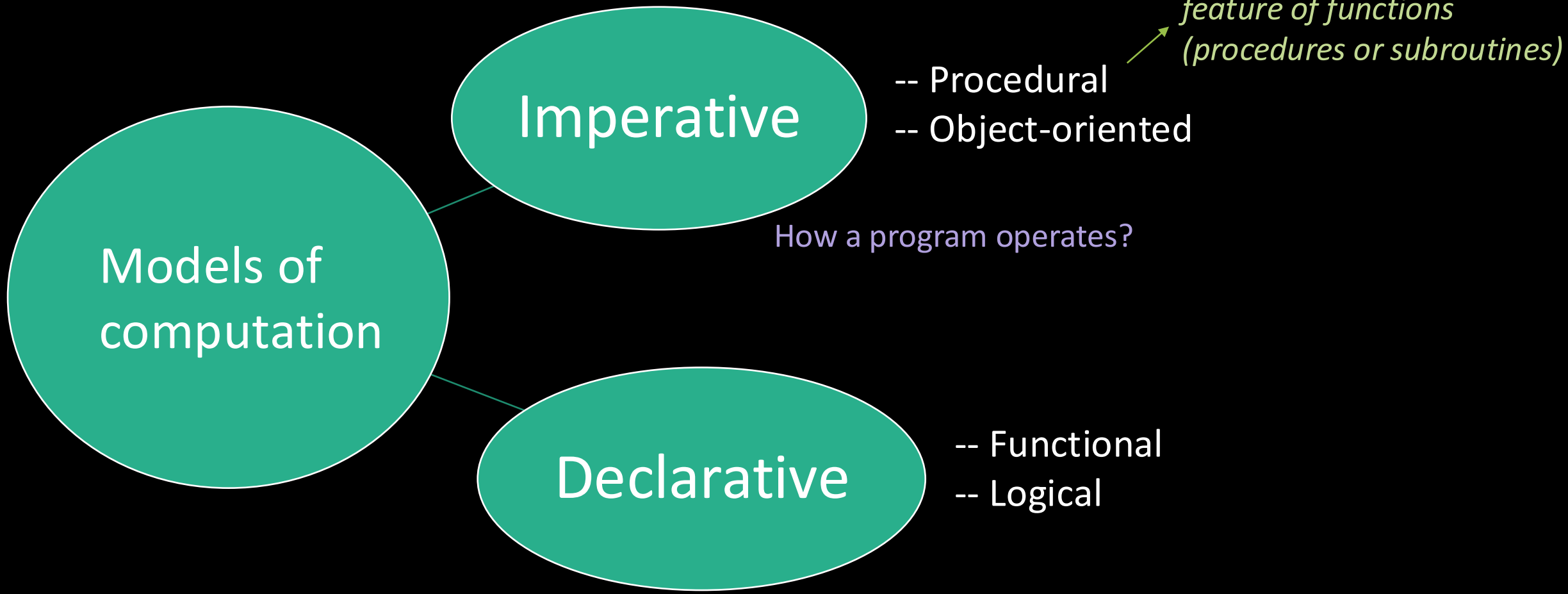
```
1- Pour flour in a bowl
2- Pour a couple eggs in the same bowl
3- Pour some milk in the same bowl
4- Mix the ingredients
5- Pour the mix in a mold
6- Cook for 35 minutes
7- Let chill
```

```
const nums = [1,4,3,6,7,8,9,2]
const result = []

for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 5) result.push(nums[i])
}

console.log(result) // Output: [ 6, 7, 8, 9 ]
```

Popular Programming Paradigms



Procedural PP

```
function pourIngredients() {
    - Pour flour in a bowl
    - Pour a couple eggs in the same bowl
    - Pour some milk in the same bowl
}

function mixAndTransferToMold() {
    - Mix the ingredients
    - Pour the mix in a mold
}

function cookAndLetChill() {
    - Cook for 35 minutes
    - Let chill
}

pourIngredients()
mixAndTransferToMold()
cookAndLetChill()
```

Functional PP

```
const nums = [1,4,3,6,7,8,9,2]

function filterNums() {
    const result = [] // Internal variable

    for (let i = 0; i < nums.length; i++) {
        if (nums[i] > 5) result.push(nums[i])
    }

    return result
}

console.log(filterNums()) // Output: [ 6, 7, 8, 9 ]
```

```
const nums = [1,4,3,6,7,8,9,2]
const result = []

for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 5) result.push(nums[i])
}

console.log(result) // Output: [ 6, 7, 8, 9 ]
```

Object-oriented PP

```
// Create the two classes corresponding to each entity
class Cook {
        constructor constructor (name) {
        this.name = name
    }

    mixAndBake() {
        - Mix the ingredients
        - Pour the mix in a mold
        - Cook for 35 minutes
    }
}

class AssistantCook {
    constructor (name) {
        this.name = name
    }

    pourIngredients() {
        - Pour flour in a bowl
        - Pour a couple eggs in the same bowl
        - Pour some milk in the same bowl
    }

    chillTheCake() {
        - Let chill
    }
}

// Instantiate an object from each class
const Frank = new Cook('Frank')
const Anthony = new AssistantCook('Anthony')

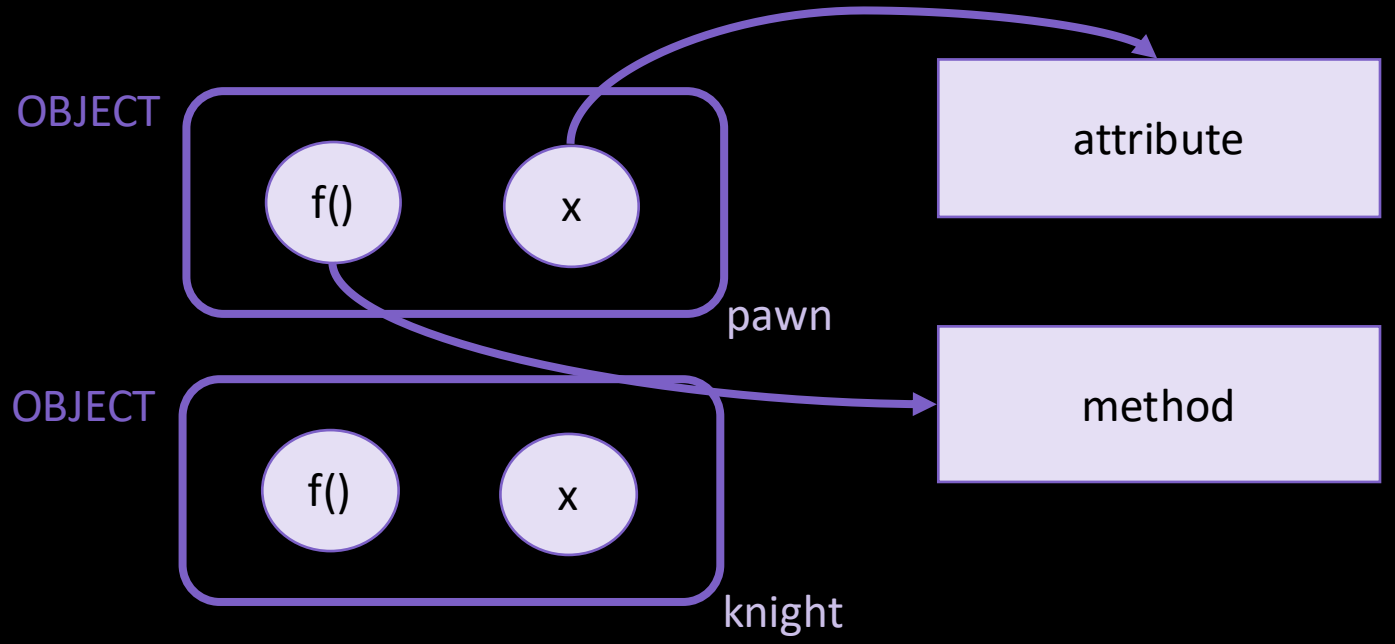
// Call the corresponding methods from each instance
Anthony.pourIngredients()
Frank.mixAndBake()
Anthony.chillTheCake()
```

What was the first OO language?

- Simula
- 1960 by researchers at the Norwegian Computing Center

What is OOP?

- is a programming paradigm based on the concept of "objects", which contain data and code: data in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods).





Example: Baking a cake

- Buy ingredients
- Mix the ingredients together with a mixer
- Pour the mixture in a cake pan
- Bake the mixture in the oven for 30 minutes on 200∘C







mixer
mix()



cakePan
hold() shape()

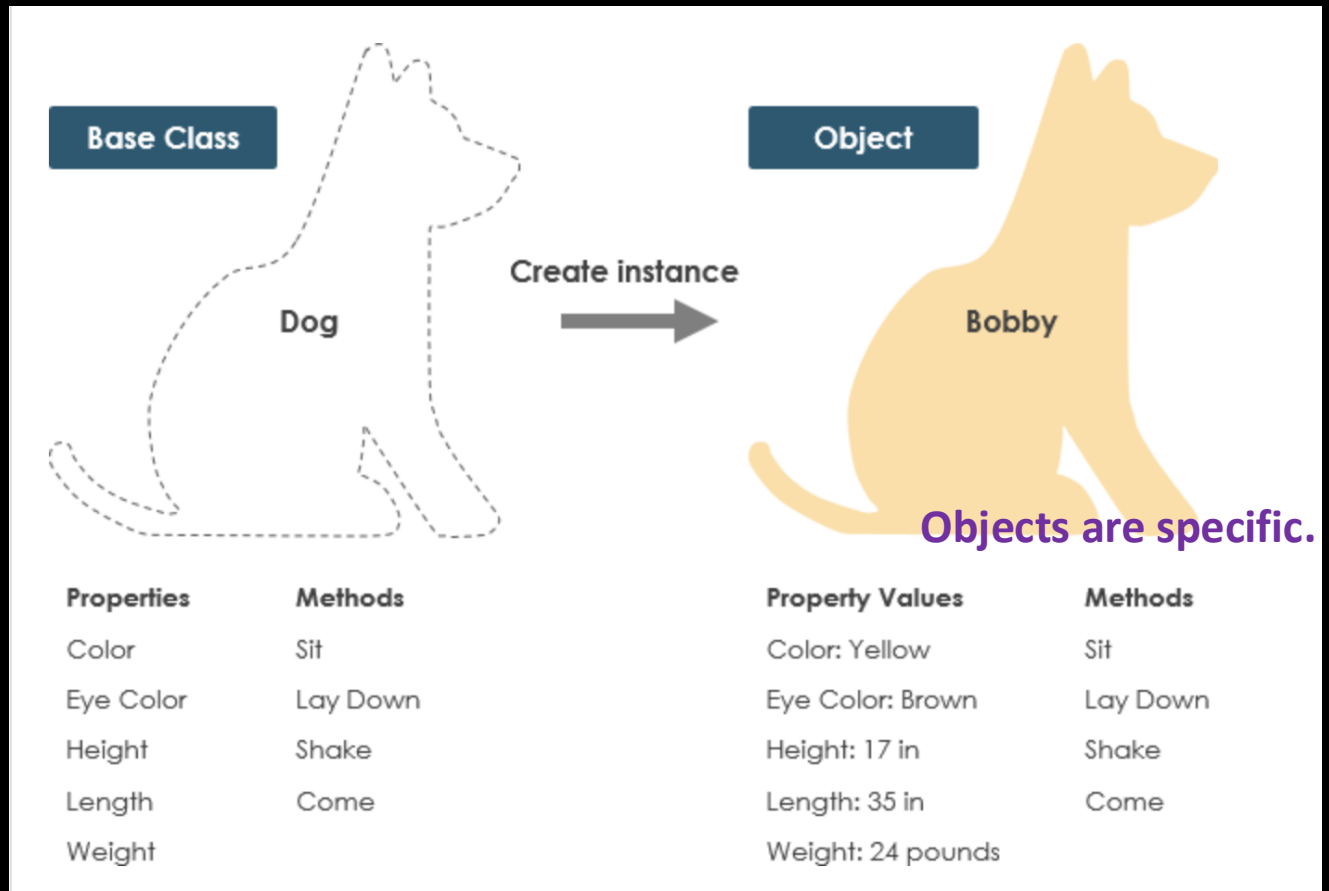


oven
bake()

Objects and classes

- OBJECTS are instances of a class. A CLASS is a template for objects.
- CLASSES describe the type of objects, while OBJECTS are usable instances of classes.

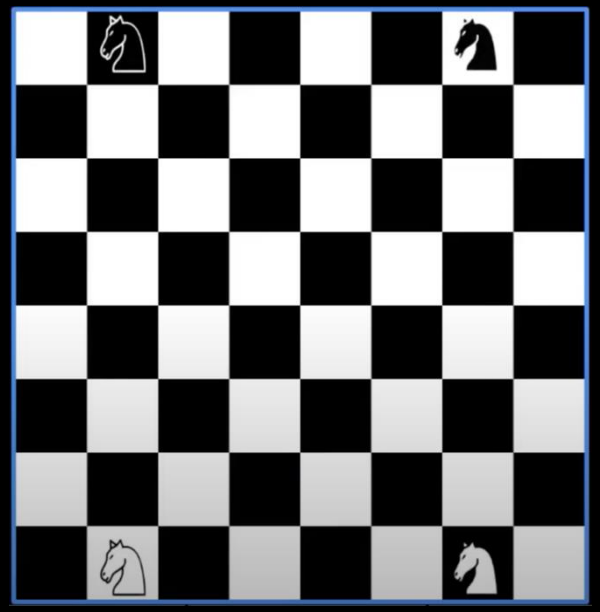
The idea of object extends to non-tangible things too (e.g., bank account, order).



The entire point of Object-Oriented Design is not about objects, it's about classes, because we use classes to create objects.

Objects and classes

a **knight object** represents only **one singular knight**



attributes

method

knight

color
position

move()

whiteKnight

color: white
startPosition: left

move()

blackKnight

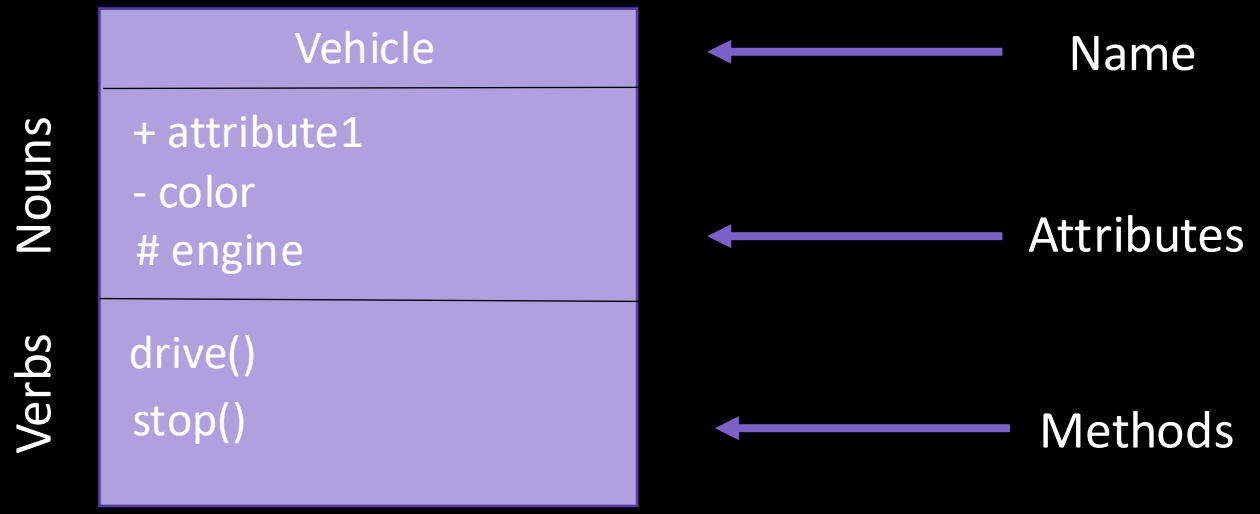
color: black
startPosition: right

move()

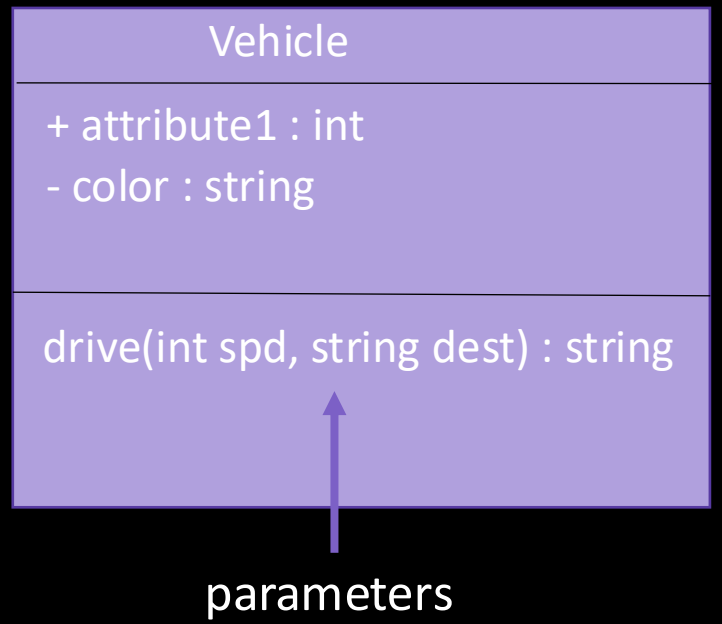
object (instance)
creating objects = instantiation

UML Class Notation

Class without signature



Class with signature

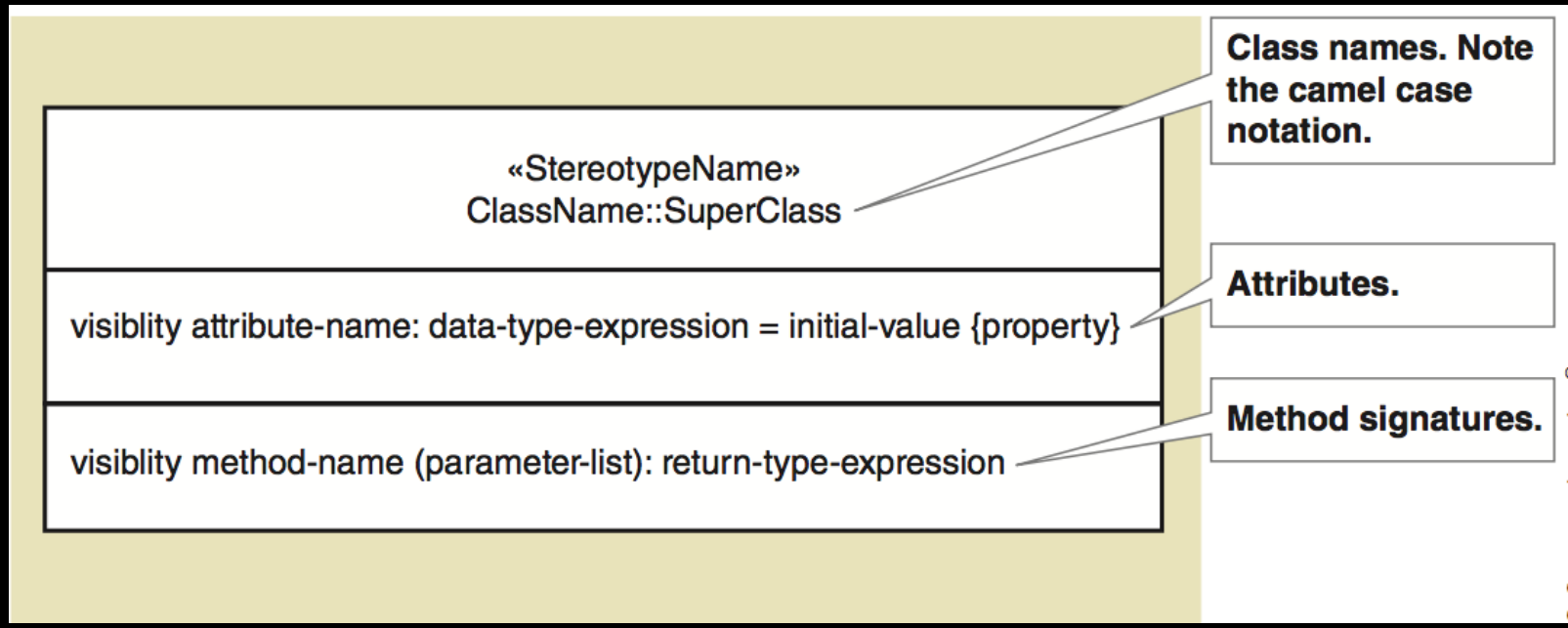


Design class diagram

The name compartment:
stereotype name, class name, and superclass name (if any).

Attributes:
- Attribute name
- Data-type-expression (such as character, string, integer, number, currency, or date)
- Initial-value, if applicable
- Property (within curly braces), such as {key}, if applicable

Method signature:
- Method name
- Method parameter list (incoming arguments)
- Return type-expression (the type of the return parameter from the method)

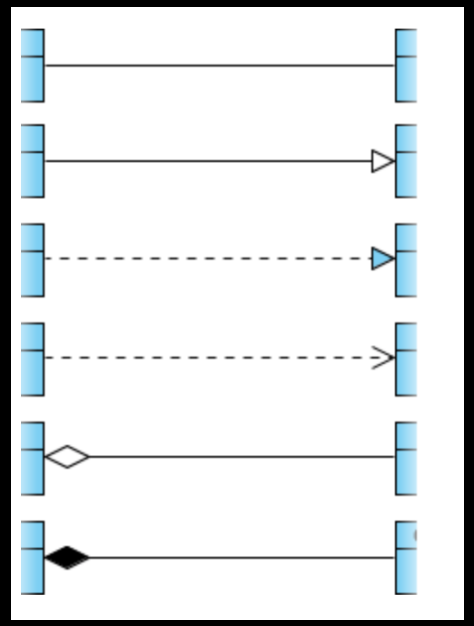


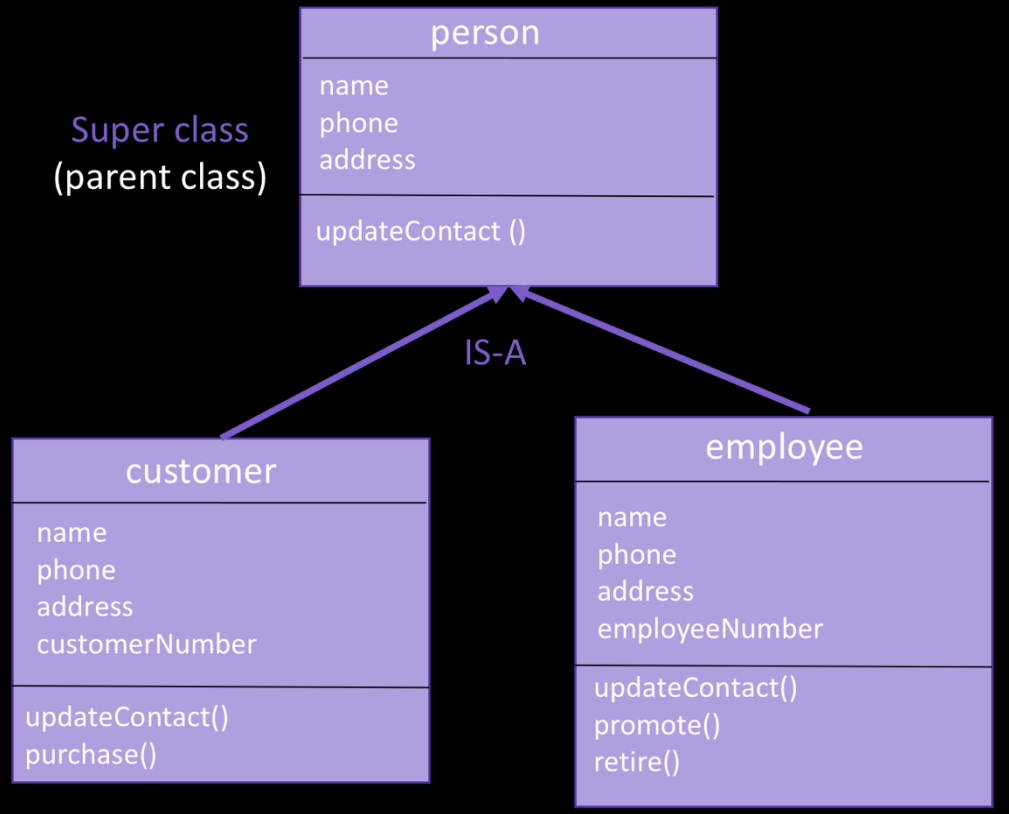
Visibility denotes whether other objects can directly access the attribute.

"+" is visible, "-" is invisible

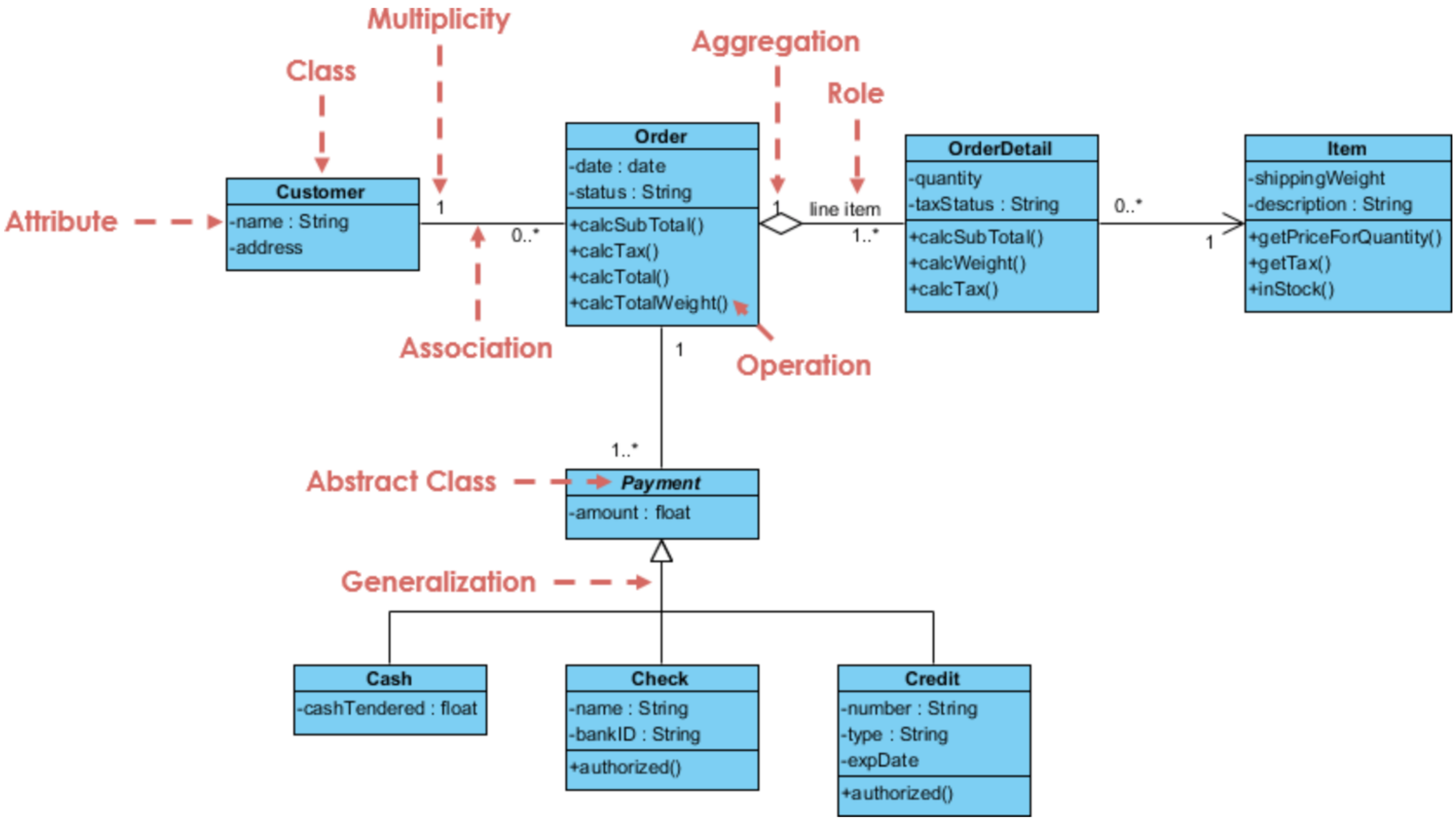
Relationships between classes

- Association
- Inheritance
- Realization
- Dependency
- Aggregation
- Composition





Class Diagram Example: Order System





Abstraction

Polymorphism
Inheritance
Encapsulation

Abstraction

refers to only showing essential details and keep everything else hidden.



BENEFITS:

- reducing complexity
- isolate impact of change

vehicle
color productionYear producer numDoors vehicleAccelerationContours vehicleSpeedContours
start() stop() drive()

vehicle
productionYear producer numDoors
start() stop() drive()

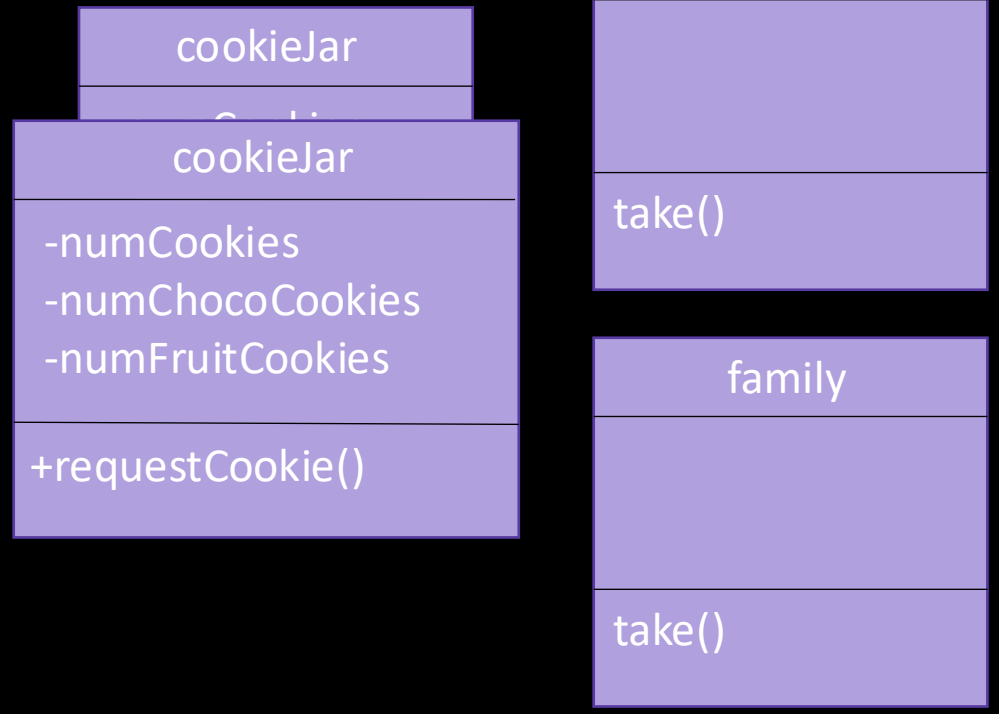
✓ Large and complex project, multiple developers work simultaneously and incrementally.

Encapsulation

refers to bundling data with methods that can operate on that data within a class.



Containing the elements of an object



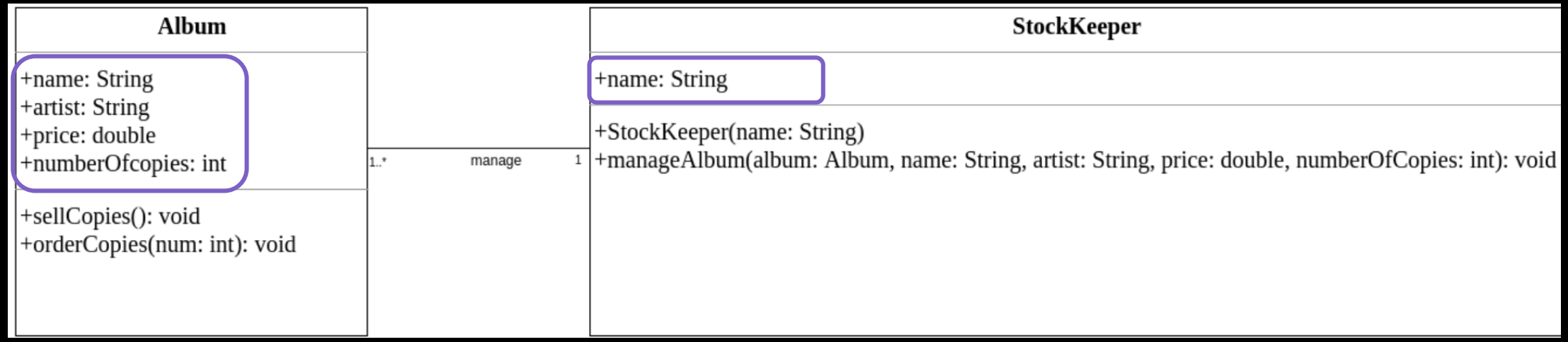
BENEFITS:

reducing complexity

reducing dependencies

increase reusability

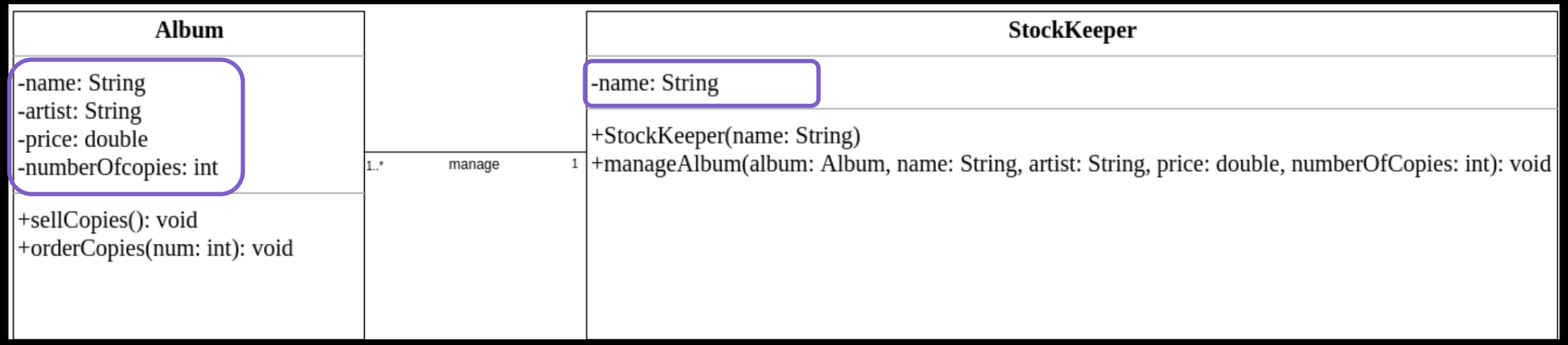
more safely change code




```java
public class Album {
    public String name;
    public String artist;
    public double price;
    public int numberOfCopies;
    public void sellCopies(){
        if(numberOfCopies > 0){
            numberOfCopies--;
            System.out.println("One album has sold!");
        }
```

```java
public class StockKeeper {
    public String name;
    public StockKeeper(String name){
        this.name = name;
    }
    public void manageAlbum(Album album, String name, String artist, double price, int
      /*
       Defining states and behaviors for album
       */
        album.name = name;
        album.artist = artist;
```

Encapsulation can be achieved by declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.




```java
public class Album {
    private String name;
    private String artist;
    private double price;
    private int numberOfCopies;
    public void sellCopies(){
        if(numberOfCopies > 0){
            numberOfCopies--;
            System.out.println("One album has sold!");
```

```java
public class StockKeeper {
    private String name;
    StockKeeper(String name){
        setName(name);
    }
    public void manageAlbum(Album album, String name, String artist, double price, int
        /*
         Defining states and behaviors for album
         */
        album.setName(name);
        album.setArtist(artist);
```

```java
class Area {

  // fields to calculate area
  int length;
  int breadth;

  // constructor to initialize values
  Area(int length, int breadth) {
    this.length = length;
    this.breadth = breadth;
  }

  // method to calculate area
  public void getArea() {
    int area = length * breadth;
    System.out.println("Area: " + area);
  }
}

class Main {
  public static void main(String[] args) {

    // create object of Area
    // pass value of length and breadth
    Area rectangle = new Area(5, 6);
    rectangle.getArea();
  }
}
```

Example Encapsulation

The main purpose of this class is to calculate the area.

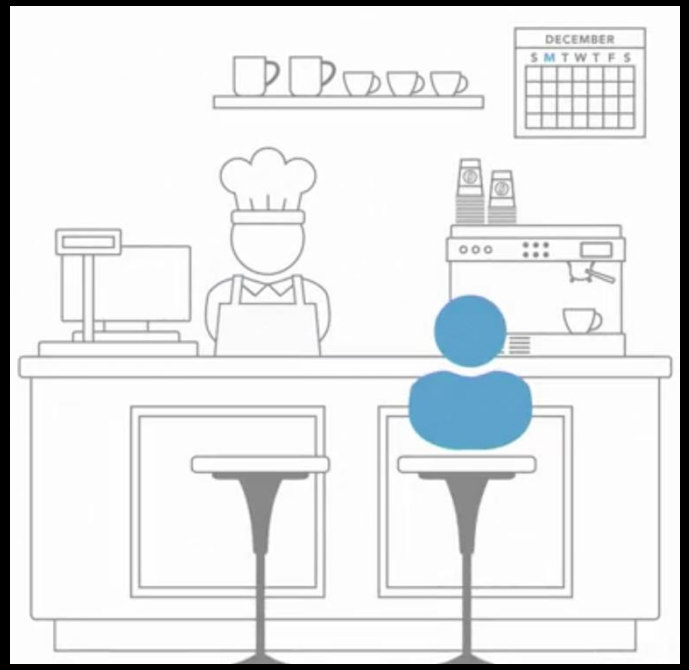
What do you need to calculate an area?

A constructor is a method having the same name as that of the class and is used to initialize the instance variable of the objects.

Encapsulation helps us to keep related fields and methods together.

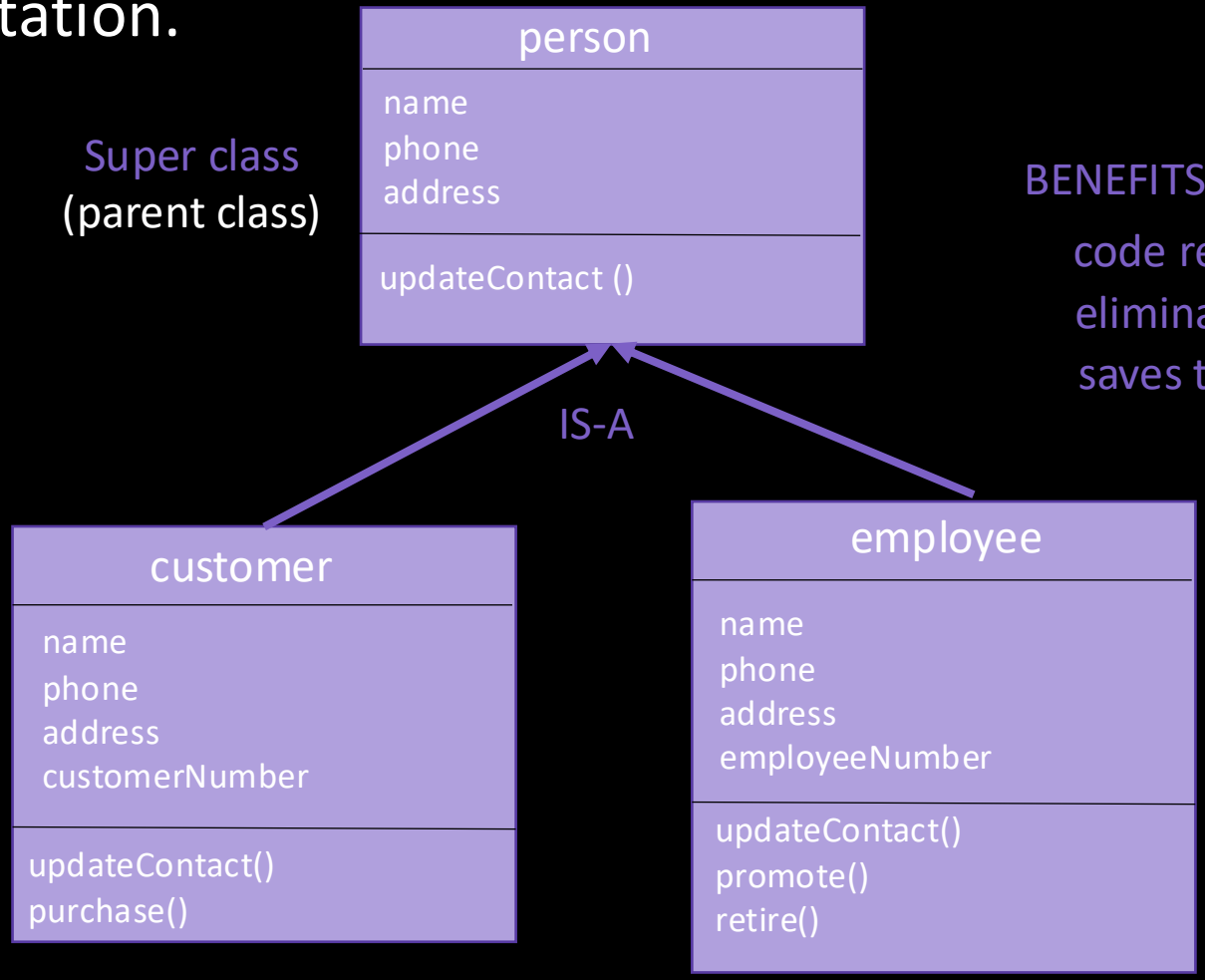
Inheritance

enables a new class to inherit the attributes and methods of existing classes using the same implementation.



A BAKERY

Super class
(parent class)



BENEFITS:

- code re-usability
- eliminate redundant code
- saves time

Subclasses
(child class)

```python
#-------------------------------------------------------------
# Define the Cake class (base class)
#-------------------------------------------------------------
class Cake:
    def __init__(self, flavor, size, frosting):
        self.flavor = flavor
        self.size = size
        self.frosting = frosting

    def bake(self):
        print(f"Baking the {self.size} {self.flavor} cake...")

    def add_frosting(self):
        print(f"Adding {self.frosting} frosting to the cake...")


#-------------------------------------------------------------
# Derived class: ChocolateCake (inherits from Cake)
#-------------------------------------------------------------
class ChocolateCake(Cake):
    def __init__(self, size):
        # Call the constructor of the base class (Cake)
        super().__init__("chocolate", size, "chocolate ganache")

    def add_extra_chocolate(self):
        print("Adding extra chocolate chips to the chocolate cake...")
```

```
#-------------------------------------------------------------
# Derived class: VanillaCake (inherits from Cake)
#-------------------------------------------------------------
class VanillaCake(Cake):
    def __init__(self, size):
        # Call the constructor of the base class (Cake)
        super().__init__("vanilla", size, "vanilla frosting")

    def add_sprinkles(self):
        print("Adding colorful sprinkles to the vanilla cake...")

#-------------------------------------------------------------
# Creating objects of the derived classes
#-------------------------------------------------------------
chocolate_cake = ChocolateCake("medium")
vanilla_cake = VanillaCake("small")

#-------------------------------------------------------------
# Using the objects
#-------------------------------------------------------------
chocolate_cake.bake()
chocolate_cake.add_frosting()
chocolate_cake.add_extra_chocolate()

vanilla_cake.bake()
vanilla_cake.add_frosting()
vanilla_cake.add_sprinkles()
```

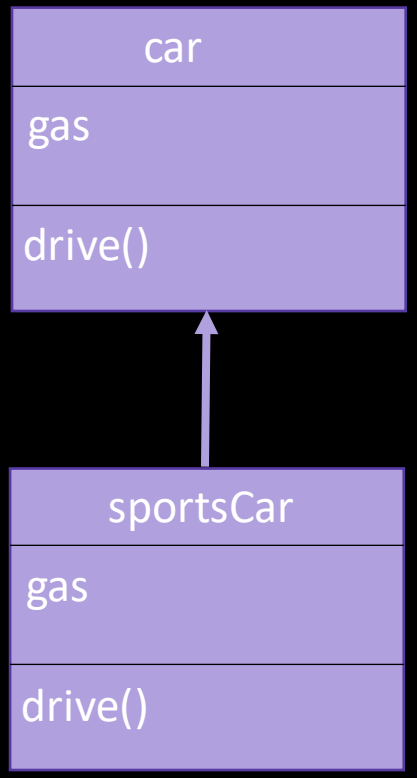
Polymorphism

perform a single action in different ways.

MAN:
- father
- husband
- lover

describes methods that can take many forms.

Dynamic (runtime)



{car.gas - = 0.2 * kilometer}

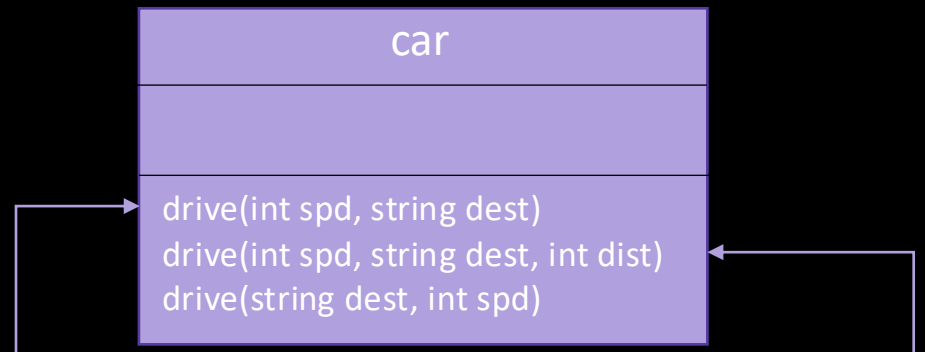
mySportsCar.drive()

OVERRIDING

{car.gas - = 0.5 * kilometer}

- implementation of a method depends on a **class hierarchy**

Static (compile time)



myCar.drive(55, ”work”)
myCar.drive(55, ”school”, 20)
myCar.drive(”work”, 55)

OVERLOADING

```java
class Square extends Polygon {

  // renders Square
  public void render() {
    System.out.println("Rendering Square...");
  }
}

class Circle extends Polygon {

  // renders circle
  public void render() {
    System.out.println("Rendering Circle...");
  }
}

class Main {
  public static void main(String[] args) {

    // create an object of Square
    Square s1 = new Square();
    s1.render();

    // create an object of Circle
    Circle c1 = new Circle();
    c1.render();
  }
}
```

Example Polymorphism

superclass: Polygon
subclasses: Square and Circle

what is it about the method?

The process of rendering a square is different than the process of rendering a circle. Hence, the render() method behaves differently in different classes.

Is the print() method an example of polymorphism?

¡Take-away!

Abstraction

- To simplify reality and focus only on the data and processes relevant to the application being built.

Encapsulation

- Data and the programs that manipulate those data are bound together and their complexity is hidden.

¡Take-away!

Inheritance

- A class can derive methods and properties from another class, resulting in a hierarchy of classes.

Polymorphism

- Different subclasses of the same superclass, can implement their shared interface in different ways. Override the inherited methods.

¡Take-away!

- ✓UML is a standardized graphical notation that helps us think what we are going to build (analysis) and how we are going to build it (design), before writing a single line of code.
- ✓OOP is a programming paradigm and not a language. It is a set of ideas and techniques that allow us to build better and more complex applications.
- ✓A PIE: Abstraction, Polymorphism, Inheritance, and Encapsulation.
- ✓How: structural and behavioral diagrams (e.g., a class diagram).
- ✓Why: All good projects (software) start with a great design!


THANK
YOU